

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 8

The Same Calls, Inside Your Own OS (xv6)

Done by:
st. gr. FAF-241, **Ciur Alexandrina**

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 Purpose and Theory **2**
1.1 Purpose 2
1.2 Short theory 2

2 Work Done **2**
2.1 Preparing the Makefile 2
2.2 Part 1 – run, inside xv6 (run.c) 2
2.3 Part 2 – pingpong (pingpong.c) 4

3 Conclusions **5**

1 Purpose and Theory

1.1 Purpose

In Lab 7 the shell's three steps – `fork`, `exec`, `wait` – were reproduced on Linux. This lab repeats the same program inside xv6, the small teaching OS built during the semester, to check that the calls are really the same when there is no glibc, only the system calls of a kernel compiled by hand. The second part makes two processes exchange a byte through pipes (`pingpong`).

1.2 Short theory

An xv6 user program is a file `user/NAME.c` that includes only `kernel/types.h` and `user/user.h`. To end up in the file system image `fs.img` it must also be listed as `$U/_NAME` in the `UPROGS` variable of the `Makefile`; otherwise the xv6 shell answers `exec NAME failed`.

`fork()` works as on Linux: 0 in the child, the child's PID in the parent. `exec(path, argv)` takes a path and a `0`-terminated argument vector; it does not search `PATH`, but the programs live in the root directory, where the shell starts. `wait(&status)` blocks until a child ends and stores directly the value the child passed to `exit`.

`pipe(p)` creates a one-way kernel buffer: `p[1]` for writing, `p[0]` for reading. `read` on an empty pipe blocks until someone writes.

2 Work Done

2.1 Preparing the Makefile

The first attempt to start the programs failed with `exec run failed`: the binaries were not in `fs.img`, because the list ended with `$U/_find`. The two new entries were added at the end of `UPROGS`; every line starts with a TAB, and only the last one has no backslash, since the `fs.img` rule follows it.

Figure 1 – End of the UPROGS list in the Makefile

```
        $U/_mystat\
        $U/_find\
        $U/_run\
        $U/_pingpong
fs.img: mkfs/mkfs README $(UPROGS)
```

2.2 Part 1 – run, inside xv6 (run.c)

`user/run.c` prints the parent's PID, forks, the child prints its own PID and becomes `echo hello from the child`; the parent waits and prints the status.

Figure 2 – run with echo

```
$ make qemu
xv6 kernel is booting

hart 1 starting
```

```
hart 2 starting
init: starting sh
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
```

Then the two lines of the child were changed so that it becomes `ls`:

Figure 3 – Change for the ls variant

```
char *args[] = { "ls", 0 };
exec("ls", args);
```

Figure 4 – run with ls

```
$ run
parent: my pid is 3
child: my pid is 4
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
kill           2 8 35512
ln             2 9 35304
ls             2 10 42880
mkdir          2 11 35568
rm             2 12 35552
sh             2 13 58312
stressfs      2 14 36416
usertests      2 15 206800
grind          2 16 51824
wc             2 17 37728
zombie         2 18 34848
logstress      2 19 37560
forphan        2 20 36352
dorphan        2 21 35800
sync           2 22 34944
mystat         2 23 36320
find           2 24 42592
run            2 25 35888
pingpong       2 26 36640
console        3 27 0
parent: child 4 ended with status 0
```

Finally the child was asked to run a program that does not exist, `exec("nosuchprog", args);`:

Figure 5 – run with nosuchprog

```
$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
```

Observations – Part 1

1. In xv6 `run.c` uses `fork()`, `exec()` and `wait()`; on Linux in Lab 7 it used `fork()`, `execlp()` and `waitpid()`. The idea and the order are identical: copy the process, turn the child into another program, wait for it in the parent. Only details differ: `exec` in xv6 receives a path and an `argv` array and does not search `PATH`, and `wait` returns the exit code directly, while on Linux it is packed in the status and read with `WEXITSTATUS`.

2. `hello from the child` is printed by `echo`, not by `run`. After a successful `exec` the child's memory holds the code of `echo`; `run` only prepared the arguments. That is also why `exec failed` does not appear in Figure 8.2. In the same way, the listing in Figure 8.4 is printed by `ls`, and it shows `run` and `pingpong` as regular files (type 2) of the file system – the proof that the `Makefile` fix worked. The last entry, `console`, has type 3: it is a device, not a file, so its size is 0. After `ls` ended, the parent printed status 0, as with `echo`.

3. The status is simply the number passed to `exit` after a failed `exec`: `exit(1)` in the xv6 `run.c`, `exit(127)` in the Lab 7 version. The kernel adds nothing; 127 is only the bash convention for "command not found" that we copied on Linux. (The PID is 4 and not 8 as in the assignment because every variant was run right after a fresh boot.)

2.3 Part 2 – pingpong (pingpong.c)

Two pipes are created: `p2c` (parent to child) and `c2p` (child to parent). Each process first closes the two ends it does not use. The four TODO lines were completed with one-byte `read(fd, &b, 1)` and `write(fd, &b, 1)` calls: the parent writes into `p2c[1]` and reads from `c2p[0]`; the child reads from `p2c[0]`, prints its message and writes the byte back into `c2p[1]`.

Figure 6 – pingpong in xv6

```
$ pingpong
6: received ping
5: received pong
```

Observations – Part 2

1. A pipe carries data in one direction only. With a single pipe both processes would read and write the same buffer: the parent could read back its own byte before the child gets it (or the child its own reply), and nothing would guarantee who receives what. With two pipes every direction has its own channel and each process reads only what the other one wrote.

2. The waiting is done by `read(c2p[0], &b, 1)` in the parent. The pipe is empty until the child writes the byte back, and `read` on an empty pipe blocks the process. So `received pong` (parent, PID 5) is always printed after `received ping` (child, PID 6). `wait(0)` comes afterwards and only collects the finished child.

3 Conclusions

The same program that was written on Linux in Lab 7 ran inside xv6 almost unchanged. `fork`, `exec` and `wait` are not features of glibc or of bash: they are system calls of the kernel, and xv6, with a few thousand lines of code, offers the same three. The differences were small and visible in the output: a different `exec` signature, a status read directly, and an exit code that is whatever the program itself passes to `exit`.

The pingpong program showed that synchronisation between processes does not need any special mechanism: a blocking `read` on a pipe is enough to make the parent wait for the child's answer, and two one-way pipes give a clean two-way conversation. The practical lesson of the lab was the build itself: a program exists for the xv6 shell only if it is listed in `UPROGS`, and one wrong character at the end of that list is enough to make `exec` fail.